Aprueba — App del alumno (Flutter, iOS + Android)

Cliente móvil de **Aprueba**, plataforma de práctica para la PAES. Implementa el catálogo de endpoints de `Aprueba_API_Backend`, el wireframe funcional (`Aprueba_App.html`) y el modelo de datos, siguiendo el Tomo I del plan.

- **Estado/navegación:** Riverpod + GoRouter
- **Red:** Dio (envelope `{data,error,meta}`, refresh transparente de tokens)
- **Caché local:** Drift (SQLite) + `flutter_secure_storage` para el refresh token
- **Integraciones reales (con claves placeholder):** Google/Apple Sign-In, Stripe (PaymentSheet), Firebase Cloud Messaging
- **i18n:** español/inglés · **Tema:** claro/oscuro (tokens del Manual de Marca)

1. Requisitos

- Flutter 3.19+ (Dart 3.3+)
- Xcode (iOS) y/o Android Studio + SDK

2. Generar las carpetas nativas y dependencias

El repo trae `lib/`, `pubspec.yaml`, `assets/` y `setup/`. Las carpetas `android/` e `ios/` se generan con Flutter sin tocar tu código:

```
cd aprueba_app
flutter create --org cl.aprueba --project-name aprueba_app .
flutter pub get
dart run build_runner build --delete-conflicting-outputs   # genera database.g.dart (Drift)
```

> `build_runner` es obligatorio: la base de datos Drift necesita `database.g.dart`.

3. Aplicar la configuración nativa

Revisa `setup/android` y `setup/ios` y aplica los fragmentos:

- **Android:** permisos (`INTERNET`, `POST_NOTIFICATIONS`), `MainActivity` heredando de `FlutterFragmentActivity` (requisito de Stripe), `minSdkVersion 23`, tema MaterialComponents, y `google-services.json` para push.
- **iOS:** `platform :ios, '13.0'`, esquema URL de Google, capabilities (Apple Sign-In, Push, Background Modes), `GoogleService-Info.plist`, `pod install`.

4. Claves de configuración

No hay secretos en el código. Pásalos por `--dart-define` (o un archivo `--dart-define-from-file`):

```
flutter run \
  --dart-define=API_BASE_URL=https://api.staging.aprueba.cl/api/v1 \
  --dart-define=STRIPE_PUBLISHABLE_KEY=pk_test_xxx \
  --dart-define=STRIPE_MERCHANT_ID=merchant.cl.aprueba \
  --dart-define=GOOGLE_SERVER_CLIENT_ID=xxx.apps.googleusercontent.com
```

Valores por defecto (placeholders) en `lib/core/config/app_config.dart`.

5. Ejecutar

```
flutter run                 # dispositivo/emulador conectado
flutter build apk           # Android
flutter build ios           # iOS (firma en Xcode)
flutter test                # pruebas unitarias (modelos)
```

6. Arquitectura

```
lib/
  core/        tema, i18n, router, red (Dio), storage seguro, widgets, config
  data/
    local/     Drift (caché + preferencias)
    models/    modelo lógico de la API (User, Question, Medals, Group, …)
    repositories/  un repositorio por dominio (patrón API + caché)
    services/  Google/Apple, Stripe, Push
  features/    una carpeta por pantalla del wireframe
  providers/   wiring Riverpod (repos, controladores, estado de sesión)
```

Flujo de datos: UI → provider → repositorio → `ApiClient` (Dio). En `GET` el repositorio escribe en la caché Drift y, ante error de red, intenta servir la última copia (`me`, `tests`, `progress`, preguntas).

Sesión: el access token (15 min) viaja en `Authorization`. Ante 401, el interceptor renueva con el refresh token (rotación) y reintenta; si falla, se marca la sesión como cerrada y el router redirige al inicio.

7. Mapa de pantallas → endpoints

Pantalla	Endpoints
Registro / Login / Social / Recuperación	`/auth/*`
Onboarding (pruebas, formato, dificultad)	`GET /tests`, `GET/PUT /me/preferences`
Home	`GET /me`, `/me/quota`, `/me/progress`
Pregunta / Resultado	`GET /practice/next`, `POST /questions/{id}/answer`
Explicación / Habilidad	`/questions/{id}/explanation`, `/questions/{id}/skill`
Recorrección	`POST /corrections`, `GET /corrections`
Cuota +5	`POST /me/quota/unlock`
Medallas / Canje / Regalo / Beneficios	`/me/medals`, `/me/medals/exchange`, `/me/gifts`, `/benefits`
Grupos / Detalle / Invitar / Stats / Compartir	`/groups*`, `/invitations/{token}/accept`

Pantalla	Endpoints
Comunidad	`/feed`, `/posts*`
Paywall / Checkout / Mi plan	`/plans`, `/checkout/sessions`, `/me/subscription*`
Ajustes / Push / Datos	`/me/settings`, `/devices`, `/me/notifications*`, `DELETE /me`

8. Notas / cambios al modelo

- Se añadió una tabla genérica de caché (`CacheEntries`) y `CachedQuestions` para repaso offline, además de `AnswerLogs` y `AppPrefs`. Es caché del cliente; no reemplaza el modelo del backend.
- El backend de la spec aún no existe en vivo: la app está lista para apuntar a `API_BASE_URL`. Sin backend, las pantallas mostrarán estados de error/caché.
- Google/Apple/Stripe/Push están integrados de verdad pero requieren las claves y la configuración nativa de la sección 3–4 para funcionar.